Lab Report No 12
Understanding of Residual Block
Artificial Intelligence

Submitted By:

[Huzaifa Shahbaz]

[21-CE-009]

[Aqsa Shah]

[21-CE-039]

Submitted to:

Engr. Hareem Khan

Dated:

17th Jun,2025

Department of Computer Engineering,

HITEC University, Taxila

Lab Task No 1:

Solution:

Brief description (3-5 lines)
You have to create a residual block with 70 layers and parameters <5m.
The code

```
import torch
import torch.nn as nn
import torch.nn.functional as F
class DepthwiseSeparableConv(nn.Module):
    """Efficient depthwise separable convolution to reduce parameters"""
    def __init__(self, in_channels, out_channels, kernel_size=3, stride=1,
padding=1):
        super().__init__()
        # Depthwise convolution
        self.depthwise = nn.Conv2d(in_channels, in_channels,
kernel_size=kernel_size,
                                   stride=stride, padding=padding, groups=in_channels,
bias=False)
        # Pointwise convolution
        self.pointwise = nn.Conv2d(in_channels, out_channels, kernel_size=1,
bias=False)
        self.bn = nn.BatchNorm2d(out_channels)
    def forward(self, x):
        x = self.depthwise(x)
        x = self.pointwise(x)
        x = self.bn(x)
        return x
class BasicResidualBlock(nn.Module):
    """Basic residual block with depthwise separable convolutions"""
    def __init__(self, channels, dropout_rate=0.1):
        super().__init__()
        self.conv1 = DepthwiseSeparableConv(channels, channels)
        self.conv2 = DepthwiseSeparableConv(channels, channels)
        self.dropout = nn.Dropout2d(dropout_rate)
        self.relu = nn.ReLU(inplace=True)
    def forward(self, x):
        residual = x
        out = self.relu(self.conv1(x))
```

```
        out = self.dropout(out)
        out = self.conv2(out)
        out += residual  # Skip connection
        out = self.relu(out)
        return out
class ResidualBlock70Layers(nn.Module):
    """70-layer residual block with <5M parameters"""
    def __init__(self, input_channels=64, base_channels=32, num_blocks=35):
        super().__init__()
        # Input projection to reduce channels early
        self.input_proj = nn.Sequential(
            nn.Conv2d(input_channels, base_channels, kernel_size=1, bias=False),
            nn.BatchNorm2d(base_channels),
            nn.ReLU(inplace=True)
        )
        # Stack of residual blocks (35 blocks × 2 conv layers = 70 layers)
        self.residual_blocks = nn.ModuleList([
            BasicResidualBlock(base_channels, dropout_rate=0.1)
            for _ in range(num_blocks)
        ])
        # Optional output projection
        self.output_proj = nn.Sequential(
            nn.Conv2d(base_channels, input_channels, kernel_size=1, bias=False),
            nn.BatchNorm2d(input_channels)
        )
        # Global skip connection
        self.global_skip = True
    def forward(self, x):
        # Store input for global skip connection
        identity = x
        # Initial projection
        x = self.input_proj(x)
        # Pass through all residual blocks
        for block in self.residual_blocks:
            x = block(x)
        # Output projection
        x = self.output_proj(x)
        # Global skip connection
        if self.global_skip and x.shape == identity.shape:
            x = x + identity
```

```
        return x
def count_parameters(model):
    """Count total trainable parameters"""
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
# Create and test the model
def create_model_and_test():
    # Create model
    model = ResidualBlock70Layers(input_channels=64, base_channels=32)
    # Count parameters
    total_params = count_parameters(model)
    print(f"Total parameters: {total_params:,} ({total_params/1e6:.2f}M)")
    # Test with sample input
    x = torch.randn(1, 64, 32, 32)  # Batch=1, Channels=64, Height=32, Width=32
    with torch.no_grad():
        output = model(x)
    print(f"Input shape: {x.shape}")
    print(f"Output shape: {output.shape}")
    print(f"Model has 70 convolutional layers across 35 residual blocks")
    return model
# Alternative even more parameter-efficient version
class UltraEfficientResidualBlock70(nn.Module):
    """Ultra-efficient 70-layer residual block"""
    def __init__(self, input_channels=64, base_channels=24, num_blocks=35):
        super().__init__()
        self.input_proj = nn.Conv2d(input_channels, base_channels, 1, bias=False)
        self.input_bn = nn.BatchNorm2d(base_channels)
        # Even smaller residual blocks
        self.blocks = nn.ModuleList()
        for i in range(num_blocks):
            block = nn.Sequential(
                nn.Conv2d(base_channels, base_channels, 3, padding=1,
groups=base_channels, bias=False),
                nn.Conv2d(base_channels, base_channels, 1, bias=False),
                nn.BatchNorm2d(base_channels),
                nn.ReLU(inplace=True),
                nn.Conv2d(base_channels, base_channels, 3, padding=1,
groups=base_channels, bias=False),
                nn.Conv2d(base_channels, base_channels, 1, bias=False),
                nn.BatchNorm2d(base_channels)
            )
```

```
            self.blocks.append(block)
        self.output_proj = nn.Conv2d(base_channels, input_channels, 1,
bias=False)
        self.output_bn = nn.BatchNorm2d(input_channels)
    def forward(self, x):
        identity = x
        x = F.relu(self.input_bn(self.input_proj(x)))
        for block in self.blocks:
            residual = x
            x = block(x) + residual
            x = F.relu(x)
        x = self.output_bn(self.output_proj(x))
        return x + identity
if __name__ == "__main__":
    print("=== Standard 70-Layer Residual Block ===")
    model1 = create_model_and_test()
    print("\n=== Ultra-Efficient 70-Layer Residual Block ===")
    model2 = UltraEfficientResidualBlock70(input_channels=64,
base_channels=20)
    params2 = count_parameters(model2)
    print(f"Ultra-efficient parameters: {params2:,} ({params2/1e6:.2f}M)")
    # Test ultra-efficient model
    x = torch.randn(1, 64, 32, 32)
    with torch.no_grad():
        out2 = model2(x)
    print(f"Ultra-efficient output shape: {out2.shape}")
```

The results (Screenshot)

```
=== Standard 70-Layer Residual Block ===
Total parameters: 100,608 (0.10M)
Input shape: torch.Size([1, 64, 32, 32])
Output shape: torch.Size([1, 64, 32, 32])
Model has 70 convolutional layers across 35 residual blocks

=== Ultra-Efficient 70-Layer Residual Block ===
Ultra-efficient parameters: 46,128 (0.05M)
Ultra-efficient output shape: torch.Size([1, 64, 32, 32])
```

Conclusion

In conclusion, the residual blocks allow nerve network to learn identification mapping, solving the fading shield problem in deep architecture. They use shortcuts (skip) connections to bypass one or more layers, preserving the necessary features. This structure enables deep networks without performance decline.

Overall, residual blocks increase accuracy, stability and training efficiency in deep learning models.